FPGA Space Invaders

Implementation manual — building, simulating, and running the Verilog reproduction (+ extensions) on a Terasic DE2-115

Reproduction of R. Parihar, A. Kumar, A. Mishra, “Design and Implementation of a FPGA based Game 'Space Invaders'” (IJCTA, Vol. 9, No. 44, 2016), ported from the paper's VHDL/Xilinx ISE original to Verilog, with gameplay extensions built on top for this lab project: a static title screen, three modes (Easy / Medium / Hard), randomised alien motion, fully independent aliens in Hard mode, alien return fire, a lives system, game-over and game-won screens, and a 7-segment score display.

1. What the game is

On power-up (or after KEY0) a **title screen** is shown: the game name, a row of aliens, “PRESS ANY KEY TO START” and the selected mode. It is colourful but completely static. Pressing KEY1, KEY2 or KEY3 starts the game. A single spaceship at the bottom of the 640×480 VGA screen faces 21 aliens (7 columns × 3 rows). You move the ship and fire one bullet at a time straight up; a hit clears an alien and adds 2 to the score. Two independent aliens at a time fire back, straight down, at random intervals. In Easy and Medium the formation drifts across the whole screen width.

How you win and lose

- **Win:** shoot all 21 aliens. A static green “GAME WON” screen appears and the game freezes.
- **Lives (alien bullets):** lives count 3 → 2 → 1. A hit from an alien bullet recentres the ship and costs a life; the hit at 1 life is fatal and lives shows 0 (dead).
- **Alien reaches the bottom:** if any live alien comes down to the ship's row, the game is over **immediately**, however many lives are left.
- **Game over:** a static magenta “GAME OVER” screen. KEY0 returns to the title screen from either end screen.

Modes (slide switches SW1 / SW0, read on the title screen)

Mode	Switches	How the aliens behave
EASY	SW1 down, SW0 down	A rigid formation drifting sideways and creeping down at random intervals — the original speed.
MEDIUM	SW1 down, SW0 up	The same rigid formation, about 3–4× faster; aliens shoot about 3× as often and their bullets are faster.
HARD	SW1 up (SW0 any)	No formation: every alien is independent. Each enters from the top edge at its own time, then wanders sideways and descends at its own speed inside a rectangle that covers almost the whole screen width. Aliens shoot ~4× as often as Easy.

The mode is latched when you press the start key — flipping the switches later does nothing. The title screen shows the current choice (green EASY, yellow MEDIUM, magenta HARD).

Controls (DE2-115)

Control	Action
KEY0	Reset — back to the title screen (also clears game over / game won). It does not start the game.
KEY1	Move ship right (also starts the game from the title screen)
KEY2	Move ship left (also starts the game from the title screen)
KEY3	Fire, one bullet at a time (also starts the game from the title screen)
SW1, SW0	Mode select (see above)

What you'll see on screen

- **Title:** “SPACE INVADERS” in fixed colours, an alien row, a decorative ship and bullet, the selected mode and a striped border — nothing moves.
- **In play:** black starfield with the striped magenta/yellow frame (same as the title), green and magenta aliens (two colour sets), a larger white ship (sprites drawn 2x), a white player bullet and yellow alien bullets (no pure red is used — see Section 5), “SCORE nnn / LIVES n” top-right, plus score (hex) and level on HEX0/HEX1/HEX2.
- **End screens:** “GAME WON” in green or “GAME OVER” in magenta, centred on a black screen.

2. What's in the project

File	Purpose
src/vga_controller.v	640×480@60Hz VGA timing (hsync/vsync/video_on/pixel_x/pixel_y)
src/clk_divider.v	Divides the 50MHz board clock down to the ~100Hz move_tick
src/graphics.v	All game logic: title/play/over/won FSM, screens, ship, rigid formation (Easy/Medium), independent aliens (Hard), bullets, alien fire, lives, scoring
src/main.v	Top-level module wiring the above together
src/debounce.v	Cleans up switch bounce on the push buttons and slide switches
src/hex7seg.v	4-bit hex → 7-segment decoder for the score/level display
board/de2115/de2115_top.v	DE2-115 board wrapper (VGA DAC, buttons, mode switches, HEX displays)
board/de2115/de2115_pins.qsf	Quartus pin + I/O-standard assignments for the DE2-115
sim/tb_main.v	Five testbenches: tb_main (gameplay, title, game won), tb_touch (alien reaches the bottom), tb_lives (bullets vs lives), tb_modes (Easy vs Medium), tb_hard (independent aliens)
docs/README.md	Full technical notes: paper citations, every deviation/extension, parameters

3. Building and running it on the DE2-115

You'll need Quartus (Prime Lite is fine for the Cyclone IV E) and the USB-Blaster driver for the DE2-115 installed.

Step 1: Create a new Quartus project

File → New Project Wizard. When asked for the device, select family **Cyclone IV E** and part **EP4CE115F29C7**.

Step 2: Add the design files

Project → Add/Remove Files in Project, and add: src/graphics.v, src/main.v, src/vga_controller.v, src/clk_divider.v, src/debounce.v, src/hex7seg.v and board/de2115/de2115_top.v. Do **not** add the sim/ file to a synthesis project.

Step 3: Set the top-level entity

Right-click `de2115_top` in the Project Navigator (Files tab) → “Set as Top-Level Entity” (not `main` — `de2115_top` is the board wrapper that has VGA_R/G/B, KEY, SW, HEX0-2).

Step 4: Import the pin assignments

Assignments → Import Assignments…, and point it at board/de2115/de2115_pins.qsf. This sets every pin (clock, buttons, SW0/SW1, VGA and the HEX displays) and their I/O standards.

Step 5: Compile

Processing → Start Compilation. A clean run ends with “Full Compilation was successful”. Warnings about unused pins/signals are normal. Check the timing report: the design targets 50 MHz.

Step 6: Connect the board

Plug the DE2-115 in via USB-Blaster and power it on. In Tools → Programmer, make sure the USB-Blaster is detected (Hardware Setup if not).

Step 7: Program the FPGA

In the Programmer, add output_files/de2115_top.sof if it isn't listed, tick “Program/Configure” and click Start. It runs immediately but is lost on power-off.

Step 8: Connect a VGA monitor and play

Plug a VGA monitor into the VGA port. Set SW1/SW0 for the mode, press any of KEY1/KEY2/KEY3 on the title screen to start; KEY1/KEY2 move, KEY3 fires, KEY0 resets to the title.

> The pin assignments (including SW[0] = PIN_AB28 and SW[1] = PIN_AC28, and the HEX displays' active-low polarity) come from the official DE2-115 pin table. One caveat: the switch, button and HEX pins default to the “2.5 V” I/O standard, which assumes the board's JP7 jumper is at its factory-default position. If JP7 was moved to 3.3V, change those IO_STANDARD lines in de2115_pins.qsf to “3.3-V LVTTL” before importing.

4. Simulating in ModelSim (recommended before hardware bring-up)

Compiling and running the testbenches first exercises the title screen, shooting, scoring, game won, the alien-reaches-the-bottom rule, bullets vs lives, Easy-vs-Medium speed and the independent Hard-mode aliens, all without the board.

```
vlib work
vlog src/vga_controller.v
vlog src/clk_divider.v
vlog src/graphics.v
vlog src/hex7seg.v
vlog src/debounce.v
vlog src/main.v
vlog sim/tb_main.v
vsim work.tb_main
add wave -r /*
run -all
```

Then run the others the same way: `vsim work.tb_touch`, `work.tb_lives`, `work.tb_modes` and `work.tb_hard`.

Watch the Transcript for `Test Case 1b/1c, 3/4, 5, 6, 6b, 6c` PASS (tb_main), `tb_touch PASS`, `tb_lives PASS`, the four `tb_modes PASS (...)` lines and the seven `tb_hard PASS (...)` lines. The touch/lives/hard checks need a good while of simulated time — let run -all finish.

> No Verilog simulator was available while building this project, so nothing here has been compiled. The design was checked with a lint script, rendered images of the title / won / over screens, and a tick-level Python model of the Hard-mode aliens using the real per-alien LFSRs (bounds never violated, aliens always spawn on the top edge, never all step together, first unshot touch at ~37 s). Do a first compile in ModelSim and Quartus yourself — and check 50 MHz timing, since the screen logic and 21 per-alien comparators sit in front of the colour register.

5. If something doesn't look right

- **No picture at all:** make sure de2115_top (not main) is the top-level entity, and try the monitor with another source.
- **Shimmering/ghosting:** change `assign VGA_CLK = CLOCK_50;` to `assign VGA_CLK = ~CLOCK_50;` in de2115_top.v and recompile.
- **Stuck on the title screen:** KEY0 is reset and can't start the game — press KEY1, KEY2 or KEY3, with KEY0 released.
- **Wrong mode:** the switches are read on the title screen and latched at the start press — set them before starting. If they read backwards or never change, check the SW pin assignments and the JP7 I/O standard.
- **Some text/colours missing (letters of the title, MODE HARD, GAME OVER):** the monitor/cable is not showing the red channel (red turns black, yellow turns green, white turns cyan). The design avoids pure red for anything important, but try another VGA cable or monitor; the VGA_R pins in the .qsf match the DE2-115 pin table.
- **Ship too jumpy:** raise SHIP_MOVE_DIVIDER in graphics.v (default 5).

- **Hard mode too easy/hard:** tune HARD_D_MIN0..2 (descent), HARD_H_MIN0..2 (sideways), HARD_FIRE_MIN and HARD_ABULLET_SPEED in graphics.v.
- **HEX displays blank or garbage:** check the JP7 note in Section 3.
- **Missing module at compile:** make sure all 7 .v files listed in Section 2 (excluding sim/) are in the project.